```
        for ch in word:
            if ch.isalpha() and ch in vowels:
                if not prev_is_vowel:
                    count += 1
                prev_is_vowel = True
            else:
                # Reset vowel group when non-vowel encountered
                if ch.isalpha():
                    prev_is_vowel = False

        # If no vowel group found, treat as 1 to avoid 0 syllables
        return count if count > 0 else 1

    def analyze_text(paragraph: str):
        stop_words = {
            "the", "is", "a", "an", "in", "of", "and", "to", "for"
        }

        # Normalize newlines and spaces
        text = paragraph.strip()

        # Paragraphs: count non-empty lines
        raw_lines = [ln.strip() for ln in paragraph.splitlines()]
        paragraphs = sum(1 for ln in raw_lines if ln)

        if paragraphs == 0:
            paragraphs = 1  # fallback if input is a single line but blank-like

        # Sentence splitting based on . ! ?
        # Requirement: sentence ends with . ! or ?
        sentence_endings = re.findall(r'[.!?]+', text)
        sentences = 0
        if text:
            # Split into candidate sentences and count those containing end punctuation
            parts = re.split(r'(?<=[.!?])\s+', text)
            sentences = sum(1 for p in parts if p.strip())
            # If above counts include fragments without punctuation, tighten:
            # But typically the sample format works with this.
            sentences = max(sentences, len(sentence_endings)) if len(sentence_endings) > 0 else sentences
        sentences = sentences if sentences > 0 else 1

        # Tokenize words: handle punctuation attached to words
        words = re.findall(r"[A-Za-z]+(?:'[A-Za-z]+)?", text.lower())
        total_words = len(words)

        if total_words == 0:
            total_words = 0
            avg_word_length = 0.0
            avg_sentence_length = 0.0
            top5 = []
            flesch = 0.0
            return total_words, sentences, paragraphs, avg_word_length, avg_sentence_length, top5, flesch

        # Word length average
```



```
        total_word_len = sum(len(w) for w in words)
        avg_word_length = total_word_len / total_words

        # Average sentence length (words per sentence)
        avg_sentence_length = total_words / sentences

        # Frequency of non-stop words
        filtered = [w for w in words if w not in stop_words]
        freq = Counter(filtered)

        # Top 5 most frequent words (sorted by frequency desc, then lexicographically)
        top5_items = sorted(freq.items(), key=lambda x: (-x[1], x[0]))
        top5 = top5_items[:5]

        # Syllables approximation: vowel groups
        total_syllables = 0
        for w in words:
            total_syllables += vowel_groups(w)

        # Flesch Reading Ease:
        # 206.835 - 1.015*(words/sentences) - 84.6*(syllables/words)
        words_per_sentence = total_words / sentences
        syllables_per_word = total_syllables / total_words
        flesch = 206.835 - 1.015 * words_per_sentence - 84.6 * syllables_per_word

        return total_words, sentences, paragraphs, avg_word_length, avg_sentence_length, top5, flesch

    def format_output(result):
        total_words, sentences, paragraphs, avg_word_length, avg_sentence_length, top5, flesch = result

        # Format top 5 as word(count)
        if top5:
            top5_str = ", ".join([f"{w}({c})" for w, c in top5])
        else:
            top5_str = ""

        print(f"Words: {total_words} | Sentences: {sentences} | Paragraphs: {paragraphs}")
        print(f"Avg Word Length: {avg_word_length:.2f} | Avg Sentence Length: {avg_sentence_length:.2f}")
        print(f"Top 5 Words: {top5_str}")
        print(f"Flesch Reading Ease: {flesch:.1f} (Standard)")

    def main():
        paragraph = input().strip()
        result = analyze_text(paragraph)
        format_output(result)

    if __name__ == "__main__":
        main()
```

INPUT

```
python is a powerful programming language
```

OUTPUT

```
Words: 6 | Sentences: 1 | Paragraphs: 1
Avg Word Length: 6.00 | Avg Sentence Length: 6.00
Top 5 Words: language(1), powerful(1), programming(1), python(1)
Flesch Reading Ease: 31.5 (Standard)
```

Q3. String Compression and Run-Length Encoding

CODE

```
def compress_rle(s: str) → str:
    if not s:
        return ""
    res = []
    i = 0
    n = len(s)

    while i < n:
        ch = s[i]
        cnt = 1
        i += 1
        while i < n and s[i] == ch:
            cnt += 1
            i += 1

        # Single characters without count still output as single char
        if cnt == 1:
            res.append(ch)
        else:
            res.append(f"{ch}{cnt}")

    compressed = "".join(res)
    # If not shorter, return original
    return compressed if len(compressed) < len(s) else s


def decompress_rle(encoded: str) → str:
    if not encoded:
        return ""

    res = []
    i = 0
    n = len(encoded)

    while i < n:
        ch = encoded[i]
        i += 1

        # Read digits after the character (if any)
        if i < n and encoded[i].isdigit():
            num = []
            while i < n and encoded[i].isdigit():
                num.append(encoded[i])
                i += 1
            cnt = int("".join(num))
        else:
            cnt = 1  # single character without count

        res.append(ch * cnt)

    return "".join(res)


def solve():
    s = input().rstrip("\n")
    compressed = compress_rle(s)

    # For decompression, we need the true compressed form.
    # If compression isn't beneficial, compressed == original.
    # Decompressing the original would still work only if it was valid RLE,
    # so we must regenerate the actual RLE first.
    # Therefore, generate the raw RLE always:
    if not s:
        raw_rle = ""
    else:
        raw = []
        i = 0
```



```
        while i < len(s):
            ch = s[i]
            cnt = 1
            i += 1
            while i < len(s) and s[i] == ch:
                cnt += 1
                i += 1
            if cnt == 1:
                raw.append(ch)
            else:
                raw.append(f"{ch}{cnt}")
        raw_rle = "".join(raw)

    decompressed = decompress_rle(raw_rle)

    print("Compressed:", compressed)
    print("Decompressed:", decompressed)


if __name__ == "__main__":
    solve()
```

INPUT

```
aabbccddddee
```

OUTPUT

```
Compressed: a2b2c2d4e2
Decompressed: aabbccddddee
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: CO2_AT3_Coding Challenge • Student Submissions Report

Q2. Vowel and Consonant Classifier with Statistics

CODE

```
def classify_sentence(s: str):
    vowels = set("aeiou")

    s = s.lower()
    vowel_count = 0
    consonant_count = 0

    vowel_freq = {v: 0 for v in vowels}
    consonant_freq = {}

    for ch in s:
        if ch.isalpha():  # ignore spaces and digits automatically
            if ch in vowels:
                vowel_count += 1
                vowel_freq[ch] += 1
            else:
                consonant_count += 1
                consonant_freq[ch] = consonant_freq.get(ch, 0) + 1

    # Most frequent vowel (handle no vowels)
    most_freq_vowel = None
    most_vowel_num = 0
    for v, cnt in vowel_freq.items():
        if cnt > most_vowel_num:
            most_vowel_num = cnt
            most_freq_vowel = v

    # Least frequent consonant (handle no consonants)
    least_freq_consonant = None
    least_consonant_num = 0
    if consonant_freq:
        least_freq_consonant = min(consonant_freq, key=consonant_freq.get)
        least_consonant_num = consonant_freq[least_freq_consonant]

    # Vowel-to-Consonant ratio (handle zero consonants)
    if consonant_count == 0:
        ratio = 0.00
    else:
        ratio = round(vowel_count / consonant_count, 2)

    # Graceful output
    if most_freq_vowel is None or most_vowel_num == 0:
        most_vowel_text = "None (0)"
    else:
        most_vowel_text = f"{most_freq_vowel} ({most_vowel_num})"

    if least_freq_consonant is None:
        least_consonant_text = "None (0)"
    else:
        least_consonant_text = f"{least_freq_consonant} ({least_consonant_num})"

    print(f"Vowels: {vowel_count} | Consonants: {consonant_count} "
          f"Most frequent vowel: {most_vowel_text} "
          f"Least frequent consonant: {least_consonant_text} "
          f"Vowel-to-Consonant Ratio: {ratio}")


# Input/Output
sentence = input().strip()
classify_sentence(sentence)
```

INPUT

```
python programming
```

OUTPUT

```
Vowels: 4 | Consonants: 13 Most frequent vowel: o (2) Least frequent consonant: y (1) Vowel-to-Consonant Ratio: 0.31
```


SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: CO2_AT3_Coding Challenge • Student Submissions Report

Q3. String Compression and Run-Length Encoding

CODE

```
def compress_rle(s: str) -> str:
    if not s:
        return ""
    res = []
    i = 0
    n = len(s)

    while i < n:
        ch = s[i]
```

SIMATS Engineering • CSE • CSA0802 — Python Programming • 29-07-2026 • Category: CO2_AT3_Coding Challenge • Student Submissions Report

Student 39 **Thanikonda Aditya** 192565068RC 4 / 4 submitted

Q1. String Character Frequency Counter

CODE

```
s = input("Enter a string: ")

s = s.lower()
freq = {}

for ch in s:
    if ch == ' ':
        continue
    freq[ch] = freq.get(ch, 0) + 1

for ch in sorted(freq.keys()):
    print(f"{ch} → {freq[ch]}")
```

INPUT

```
Hello World
```

OUTPUT

```
Enter a string: d -> 1
e -> 1
h -> 1
l -> 3
o -> 2
r -> 1
w -> 1
```